新規フォーマットフォルダ作成モーダル UI仕様書

1. 概要

1.1 コンポーネントの目的

新規フォーマットフォルダ作成モーダルは、部門直下に新しいフォーマットフォルダを作成するためのモーダルダイアログです。material、condition、resultなどのフォーマットフォルダを作成します。

1.2 使用場面

- **表示場所**: フォーマット管理画面のモーダルオーバーレイ
- **トリガー**:「新規フォーマットフォルダ作成」ボタンのクリック
- **主な用途**:
  - XMLテンプレート用のフォーマットフォルダ作成（material/condition/result）
  - MAIMLワークフロー用のフォーマットフォルダ作成
  - 部門配下のルートフォルダ作成

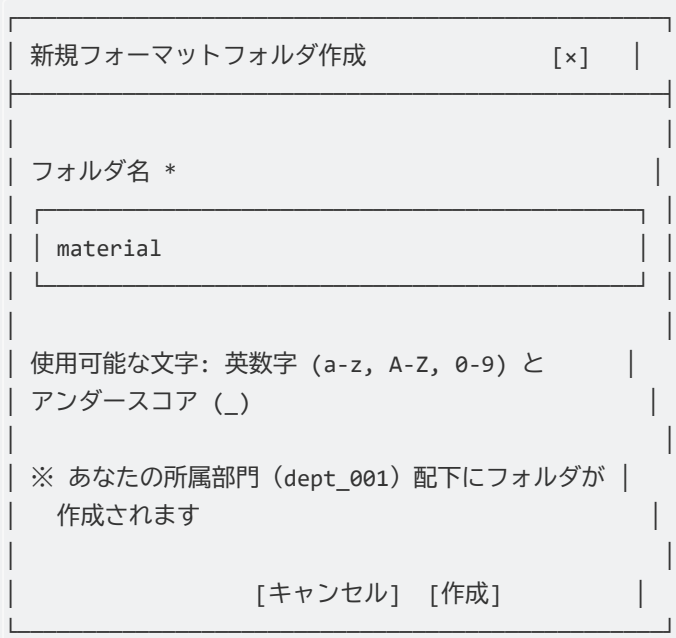
1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: 親コンポーネント（FormatManagement）で認証済み

---

2. 画面構成

2.1 モーダル構造



2.2 主要UI要素

2.2.1 モーダルヘッダー

- **タイトル**: 「新規フォーマットフォルダ作成」

2.2.2 フォルダ名入力フィールド

- **タイプ**: Input
- **ラベル**: 「フォルダ名」
- **バリデーション**:
  - 必須
  - 正規表現: `/^[a-zA-Z0-9_]+$/` （英数字とアンダースコアのみ）
  - 最小1文字
  - 最大255文字
- **プレースホルダー**: 「フォルダ名を入力」

2.2.3 補足説明エリア

- **使用可能文字**: 「使用可能な文字: 英数字 (a-z, A-Z, 0-9) とアンダースコア (_)」

- **部門情報**（条件付き）:「※ あなたの所属部門（{departmentName}）配下にフォルダが作成されます」

2.2.4 エラー表示（条件付き）

- **表示条件**: APIエラー発生時
- **表示位置**: フォルダ名フィールドの下部
- **スタイル**: 赤いエラーメッセージ

2.2.5 アクションボタン

- **キャンセルボタン**: モーダルを閉じる
- **作成ボタン**: フォーマットフォルダ作成を実行（ローディング表示あり）

2.3 インタラクション

2.3.1 モーダル表示

- **操作**:「新規フォーマットフォルダ作成」ボタンをクリック
- **結果**: モーダルが開き、フォームとエラーメッセージがリセットされる

2.3.2 フォルダ名入力

- **操作**: フォルダ名を入力
- **結果**:
  - 255文字以上は入力できない
  - 不正な文字を入力するとバリデーションエラーが表示される

2.3.3 作成実行

- **操作**:「作成」ボタンをクリック
- **結果**:
  - バリデーションが実行される
  - 成功時: モーダルが閉じ、フォーマット一覧に新しいフォルダが表示される

- 失敗時: エラーメッセージが表示される

2.3.4 キャンセル

- **操作**: 「キャンセル」ボタンまたは×ボタンをクリック
- **結果**: モーダルが閉じる

---

3. 機能仕様

3.1 フォルダ名バリデーション

3.1.1 必須チェック

```
{
  required: true,
  message: 'フォルダ名を入力してください',
}
```

3.1.2 正規表現パターンチェック

```
{
  pattern: /^[a-zA-Z0-9_]+$/,
  message: '英数字とアンダースコアのみ使用できます',
}
```

- **許可される文字**:
  - 英字（大文字・小文字）: `a-zA-Z`
  - 数字: `0-9`
  - アンダースコア: `_`
- **禁止される文字**:

- スペース
- ハイフン: `-`（サブフォルダとは異なり許可しない）
- 日本語文字
- 特殊記号（`@`, `#`, `$`, `%` など）

3.1.3 文字数チェック

```
{
  min: 1,
  max: 255,
  message: 'フォルダ名は1〜255文字で指定してください',
}
```

3.1.4 UI動作

- ユーザー操作:「作成」ボタンをクリック
- 画面変化:
  - バリデーションエラーがある場合、フィールド下部に赤いエラーメッセージが表示される
  - すべてのバリデーションが通過すると、送信処理が実行される

3.2 フォーマットフォルダ作成処理

3.2.1 送信ハンドラ

```
const handleSubmit = async () => {
  try {
    // バリデーション
    const values = await form.validateFields();
    const { folderName } = values;

    setLoading(true);
    setErrorMessage(null);
```

```
    // 親コンポーネントのonSubmitを呼び出し
    await onSubmit(folderName);

    // 成功したらモーダルを閉じる
    form.resetFields();
    setErrorMessage(null);
    onCancel();
  } catch (error) {
    console.error('❌ [CreateFormatModal] エラー:', error);

    // APIエラーの場合、エラーメッセージを表示
    const apiError = error as { response?: { data?: { error?: { message?: string } } };
message?: string };
    if (apiError?.response?.data?.error?.message) {
      setErrorMessage(apiError.response.data.error.message);
    } else if (apiError?.message && apiError.message !== 'Validation failed') {
      setErrorMessage(apiError.message);
    }
    // フォームバリデーションエラーの場合は何もしない（Form内で表示される）
  } finally {
    setLoading(false);
  }
};
```

3.2.2 データ構造

- **送信データ**: フォルダ名（文字列）のみ
- 親コンポーネントで部門情報と組み合わせて完全なパスを構築

例:

```
await onSubmit('material');
```

結果のS3パス:

```
{organization_id}/formats/{department_cd}/material/
```

3.2.3 UI動作

- ユーザー操作:「作成」ボタンをクリック
- 画面変化:

- ボタンがローディング状態になる
- 成功時: フォームがリセットされ、モーダルが閉じる
- 失敗時: エラーメッセージがフィールド下部に表示される

3.3 エラーハンドリング

3.3.1 APIエラーの表示

```
// APIエラーの場合、エラーメッセージを表示
const apiError = error as { response?: { data?: { error?: { message?: string } } }; message?:
string };
if (apiError?.response?.data?.error?.message) {
  setErrorMessage(apiError.response.data.error.message);
} else if (apiError?.message && apiError.message !== 'Validation failed') {
  setErrorMessage(apiError.message);
}
```

- **表示方法**: Form.Itemの `validateStatus` と `help` プロパティを使用
- **表示位置**: フォルダ名フィールドの下部
- **スタイル**: 赤い枠線 + 赤いエラーメッセージ

3.3.2 バリデーションエラー

- **表示方法**: Ant Design Formの標準バリデーション
- **表示位置**: フィールド下部
- **スタイル**: 赤い枠線 + 赤いエラーメッセージ

3.3.3 UI動作

- ユーザー操作: なし（エラー発生時に自動表示）
- 画面変化:
  - フィールド下部にエラーメッセージが表示される
  - フィールドの枠線が赤くなる

3.4 フォーム自動リセット

3.4.1 モーダル表示時のリセット

```
useEffect(() => {
  if (open) {
    form.resetFields();
    setErrorMessage(null);
  }
}, [open, form]);
```

- **目的**: モーダルが開くたびにフォームとエラーメッセージを初期状態に戻す
- **タイミング**: `open` プロパティが `true` になった時

3.4.2 UI動作

- ユーザー操作: モーダルを開く
- 画面変化:
  - フォームフィールドがクリアされる
  - エラーメッセージがクリアされる
  - 前回入力した内容が残らない

---

4. データ仕様

4.1 Props定義

```
export interface CreateFormatModalProps {
  open: boolean;                                    // モーダル表示状態
  onCancel: () => void;                             // キャンセル時のコールバック
  onSubmit: (folderName: string) => Promise<void>;  // 送信時のコールバック
```

```
  departmentName?: string;                    // 部門名（表示用）
}
```

Props説明:

Prop名	型	必須	説明
open	boolean	✅	モーダルの表示状態（true=表示、false=非表示）
onCancel	() => void	✅	キャンセル時に呼び出されるコールバック関数
onSubmit	(folderName: string) => Promise	✅	送信時に呼び出されるコールバック関数（非同期）、フォルダ名を渡す
departmentName	string	-	部門名（補足説明に表示）

4.2 State定義

```
const [form] = Form.useForm();                          // Ant Design Form インスタンス
const [loading, setLoading] = useState(false);          // 送信中フラグ
const [errorMessage, setErrorMessage] = useState<string | null>(null); // エラーメッセージ
```

State説明:

State名	型	説明
form	FormInstance	Ant Design Formのインスタンス（バリデーションとリセットに使用）
loading	boolean	送信処理中かどうか（true=送信中、ボタンがローディング状態になる）
errorMessage	string | null	APIエラーメッセージ（null=エラーなし）

---

5. API連携

このコンポーネントは直接APIを呼び出しません。 親コンポーネント（FormatManagement）の `onSubmit` コールバックを呼び出し、親コンポーネントがAPI通信を行います。

5.1 呼び出されるAPI（親コンポーネント経由）

想定されるAPI:

- **POST** `/api/v1/experiment_admin/formats`
  - フォーマットフォルダ作成API
  - リクエストボディ: `{ folderName: string, departmentCd: string }`
  - レスポンス: 作成されたフォーマット情報

---

6. エラーハンドリング

6.1 エラーケースと表示方法

エラーケース	検知タイミング	表示方法	表示内容
フォルダ名未入力	バリデーション時	フィールド下部	「フォルダ名を入力してください」
フォルダ名が不正な文字	バリデーション時	フィールド下部	「英数字とアンダースコアのみ使用できます」
フォルダ名が長すぎる	バリデーション時	フィールド下部	「フォルダ名は1〜255文字で指定してください」
フォルダ名重複	API返却時	フィールド下部	APIから返却されたエラーメッセージ
API通信エラー	API返却時	フィールド下部	APIから返却されたエラーメッセージ

6.2 エラー回復方法

- **バリデーションエラー**: ユーザーが入力を修正
- **重複エラー**: ユーザーがフォルダ名を変更
- **API通信エラー**: ユーザーが再試行またはキャンセル

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**:

`frontend/src/components/project/CreateFormatModal.tsx`

7.2 関連ファイル

- **親コンポーネント**: `frontend/src/components/project/FormatManagement.tsx`
- **型定義**: このファイル内で定義・export（CreateFormatModalProps型）

7.3 依存ライブラリ

- **Ant Design**:
  - Modal
  - Form
  - Input
  - Typography (Text)
  - Space
  - theme

7.4 親子関係

```
FormatManagement (親)
  ↓ Props: open, onCancel, onSubmit, departmentName
CreateFormatModal (このコンポーネント)
```

親コンポーネントから受け取るデータ:

- モーダル表示状態
- 部門名（表示用）

親コンポーネントへの通知:

- キャンセルイベント（`onCancel()`）
- 送信イベント（`onSubmit(folderName)`）

---

8. UI/UXの考慮事項

8.1 レスポンシブデザイン

- **幅**: 500px（固定）
- **高さ**: コンテンツに応じて可変

8.2 アクセシビリティ

- **キーボード操作**: Enterキーで送信、Escキーでキャンセル
- **エラーメッセージ**: 明確で具体的なメッセージ

8.3 視覚的フィードバック

8.3.1 ローディング状態

- 送信中は「作成」ボタンがローディング表示
- モーダルは閉じられない（confirmLoadingで制御）

8.3.2 エラー表示

- **フィールドエラー**: 赤い枠線 + 下部に赤いエラーメッセージ
- APIエラーもフィールドに統合表示

8.4 ユーザーガイダンス

- **補足説明**: 使用可能な文字を明示
- **部門情報**: 作成場所を明示（所属部門配下）
- **プレースホルダー**:「フォルダ名を入力」

8.5 フォーム自動リセット

- **モーダル表示時**: フォームとエラーメッセージが自動的にリセットされる

- **送信成功時**: フォームとエラーメッセージが自動的にリセットされる

8.6 フォーム保存

- **preserve={false}**: モーダルが閉じられた時にフォームの値を保持しない
- **destroyOnHidden**: モーダルが閉じられた時にコンポーネントを破棄

---

9. 使用例

9.1 materialフォーマットフォルダを作成

```
// 親コンポーネントでの呼び出し
<CreateFormatModal
  open={isCreateFormatModalOpen}
  onCancel={() => setIsCreateFormatModalOpen(false)}
  onSubmit={handleCreateFormat}
  departmentName="dept_001"
/>

// ユーザーが「material」と入力して送信
// 結果のS3パス: {organization_id}/formats/dept_001/material/
```

9.2 conditionフォーマットフォルダを作成

```
// ユーザーが「condition」と入力して送信
// 結果のS3パス: {organization_id}/formats/dept_001/condition/
```

9.3 カスタムフォーマットフォルダを作成

```
// ユーザーが「custom_workflow」と入力して送信
// 結果のS3パス: {organization_id}/formats/dept_001/custom_workflow/
```

10. 注意事項

10.1 フォーマットフォルダとサブフォルダの違い

項目	フォーマットフォルダ（このコンポーネント）	サブフォルダ（CreateFormatFolderModal）
作成場所	部門直下	フォーマットフォルダ配下
許可文字	英数字とアンダースコア	英数字、アンダースコア、ハイフン
用途	ルートフォルダ（material/condition/result等）	階層化のためのサブフォルダ
例	`material`	`company_f`

10.2 推奨フォルダ名

- **XMLテンプレート用**: `material`, `condition`, `result`
- **MAIMLワークフロー用**: `chemistry`, `biology`, `materials_science` など

11. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を作成（Phase 4: フォーマット管理画面）	Cline